

OT-Recon

Detailed Technical Documentation

Version: Current Development Build

Overview

OT-Recon is a modular Python-based command-line tool designed for Operational Technology (OT) network reconnaissance. Rather than functioning as a generic port scanner, it discovers hosts, identifies industrial protocols, fingerprints industrial assets, classifies devices, assesses security risks, and presents structured reports suitable for OT security assessments.

Project Goals

- Discover live hosts efficiently.
- Detect OT protocols (S7, Modbus, EtherNet/IP, OPC UA, etc.).
- Fingerprint Siemens and Rockwell assets using Nmap NSE scripts.
- Classify assets (PLC, HMI, Gateway, Server, Unknown).
- Highlight exposed services and security risks.
- Provide a modular architecture for future protocol support.

Architecture

CLI → main.py → scanner.py → parser.py → enricher.py → fingerprinter.py → classifier.py → risk.py → reporter.py

Module Responsibilities

main.py: Controls workflow and CLI.

scanner.py: Performs host discovery and targeted Nmap scans.

parser.py: Parses XML into Python objects.

enricher.py: Maps ports/services to OT protocols.

fingerprinter.py: Executes protocol-specific NSE scripts (s7-info, http-title, ssl-cert, enip-info) and extracts vendor, model, firmware and evidence.

classifier.py: Uses collected evidence to identify PLCs, HMIs and other OT assets with confidence scoring.

risk.py: Flags exposed industrial services and attack surface.

reporter.py: Generates Rich CLI tables and summaries.

Workflow

1. Discover live hosts.
2. Scan only discovered hosts for selected OT/IT ports.
3. Parse Nmap XML results.
4. Enrich ports into protocols.
5. Fingerprint supported devices using NSE scripts.
6. Classify assets using evidence instead of ports alone.
7. Assess risks.
8. Present structured report.

Current Fingerprinting Support

Siemens PLC: s7-info (Module, System Name, Firmware, Module Type).

Siemens HMI: http-title + ssl-cert (SIMATIC HMI detection).

Rockwell PLC/HMI: enip-info (Vendor, Product Name, Revision, Device Type).

Why Fingerprinting?

Simple port-based classification incorrectly labels many devices. Fingerprinting collects evidence from protocol-aware scripts, enabling accurate identification of PLCs, HMIs and gateways while improving confidence scoring.

Risk Engine

Current rules identify exposed Modbus, S7, EtherNet/IP, HTTP, SSH, FTP and multiple exposed services. Planned improvements include severity scoring and firmware-aware vulnerability checks.

Future Roadmap

- Context-aware confidence scoring.
- Modbus and OPC UA deep fingerprinting.
- Passive asset discovery with Scapy.
- Network topology mapping.
- Firmware/CVE correlation.
- Export to JSON, CSV and HTML reports.

Learning Outcomes

Python packaging, CLI development, Nmap automation, XML parsing, modular software architecture, OT protocol awareness, industrial asset identification, rule-based security analysis, Git/GitHub workflows.